GEBZE TECHNICAL UNIVERSITY

Computer Engineering Department

CSE 396 Module Documentation

Çılgın Kurye

Module M1: Game Server and Rules

Instructor: Erkan Zergeroğlu **Company:** Board&Byte **Group:** Group-8 **Date:** 6 November 2026

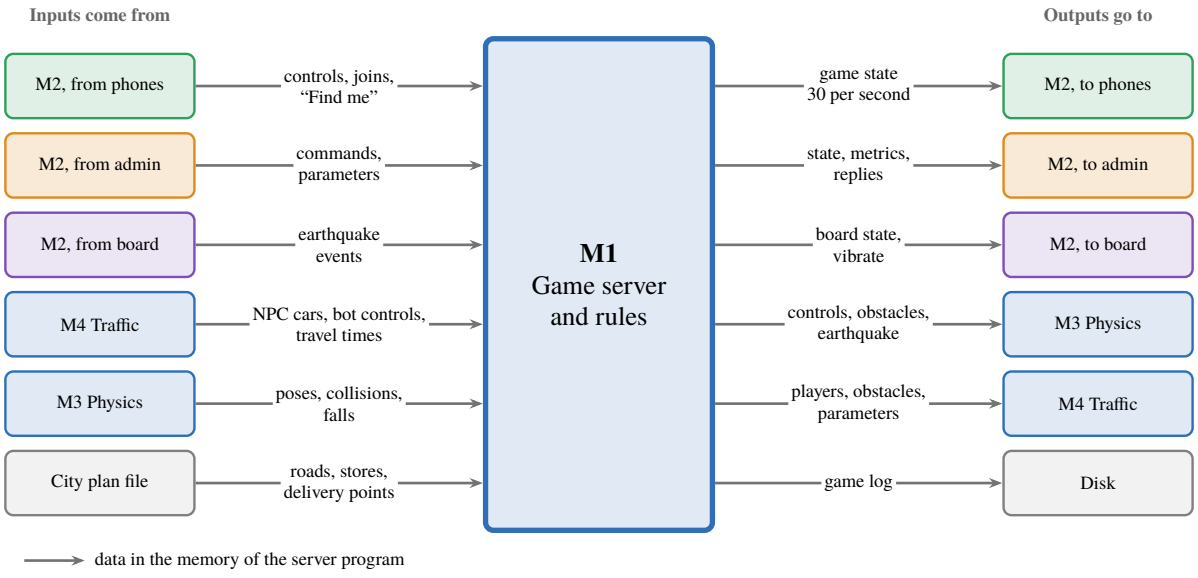
Name	Student No.	Name	Student No.
Tuğberk Akbulut	230104004070	Abdullatif Yılmaz	230104004031
Berk Andaçgil	230104004066	Selim Barut	220104004046
Umutcan Ocak	220104004903		

1. Purpose

M1 is the brain of the game. It runs on the main computer and it is the only part that can change the game. It keeps the game clock and the one true game state. It knows every team, store, package, delivery point, score, fall, obstacle and earthquake. Phones only send their buttons. M1 decides what happens, and then everybody sees the same result.

Runs on	Technology	Requirements
Main computer, in one server program together with M2, M3 and M4	TypeScript on Node.js, a fixed step game loop with 30 ticks per second, seeded random numbers, JSON files for the city plan and the game log	R1 R2 R4 R5 R6 R7 R9 R11

M1 has no network code. All its inputs and outputs are data in the memory of the server program. The figure shows which module gives M1 each kind of data and which module gets each result.



2. Requirements covered

M1 takes part in eight of the eleven system requirements. The last column says what will work when M1 is finished.

ID	Requirement	What M1 does for it	When M1 is done
R1	Multiplayer game	Lobby with 4 teams and at most 2 players in a team. Bots fill the empty places. A game is at most 5 minutes.	A full game runs from the lobby to the winner, with humans and bots together
R2	Delivery cycle	Automatic pickup at the store. Picks a delivery point by travel time, with a near or far chance per player	Every player always has a package or a point to go to, and points are fair
R4	Collisions and falls	Gives a penalty time after every fall that M3 reports	A fallen player waits and then rides on with the package
R5	No map on the phone	Puts the compass target and the "Find me" waiting time into the game state	The phone can show the arrow and the button without a map

ID	Requirement	What M1 does for it	When M1 is done
R6	Obstacles	Keeps the one obstacle list that M3, M4, M5 and M8 use	An obstacle from the admin is seen everywhere in the same tick
R7	Earthquake	One path for a hand hit, the motors and the admin button. Road work and falls inside the radius	A hit on the board changes the game for everybody
R9	Live admin panel	Checks admin commands and sends each parameter to the module that uses it	Every parameter can be changed during the game
R11	Engineering requirements	Only M1 changes the game state. Every game can be played again from the log	The same log always gives the same game

3. Inputs and outputs

The table gives the content of each input and output. Value ranges are shown only for the main control values.

Direction	Data	From / to	Content
In	Controls	M2, from phones	Throttle and brake from 0 to 1, steer from −1 to 1, a control number. “Deliver” and “Find me” come as events
In	Join	M2, from phones	Player key, name, team. M2 also reports when a player is lost or comes back
In	Admin commands	M2, from admin	Parameter changes, start, pause, resume, end, reset, new game code, obstacles, earthquake
In	Board events	M2, from board	Source (hand or motors), strength from 0 to 10, zone, confidence
In	Traffic	M4	NPC cars, bot controls, travel time table
In	Physics	M3	New motorcycle poses, collision and fall events
Out	Game state	M2, to phones and admin	Every tick: phase, time, scores, players, NPC cars, obstacles, earthquake. For each player also the compass target, the “Find me” waiting time and the number of the last control that M1 used
Out	Board state	M2, to board	Phase, players, delivery points, obstacles and earthquake in board coordinates
Out	Metrics	M2, to admin	Twice per second: the values of all admin graphs
Out	Game log	Disk	The seed and, for every tick, the controls, admin commands and board events

4. Main components

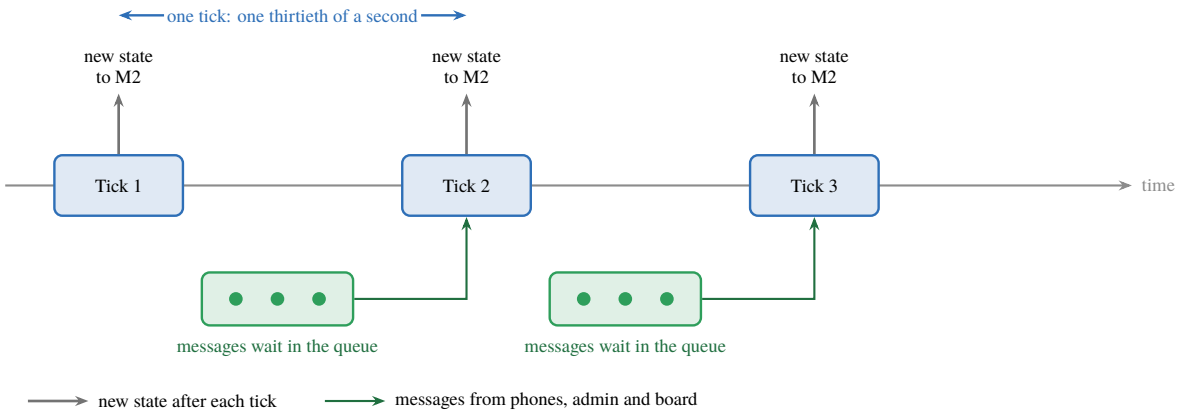
M1 is made of seven parts. Each part takes some data and gives a result to the next part or to another module.

Component	Takes	Gives
Game loop	The clock	Runs the steps of a tick in a fixed order, 30 times per second
Match and lobby	Joins, lost players, admin start and pause	Game phase, teams, bots in empty places
Delivery and scoring	Poses, “Deliver” events, travel times	Packages, delivery points, points, a reason when a delivery is refused
Penalties	Fall events from M3	Penalty time, then a stand up request to M3
Obstacles and earthquake	Admin obstacles, board events	Obstacle list with a version number, earthquake centre and radius
Admin and parameters	Admin commands	Checked values sent to the owner module, ok or error replies
State builder and recorder	The world after the tick	Frozen game state, board state, metrics with a short history, the game log

5. How it works

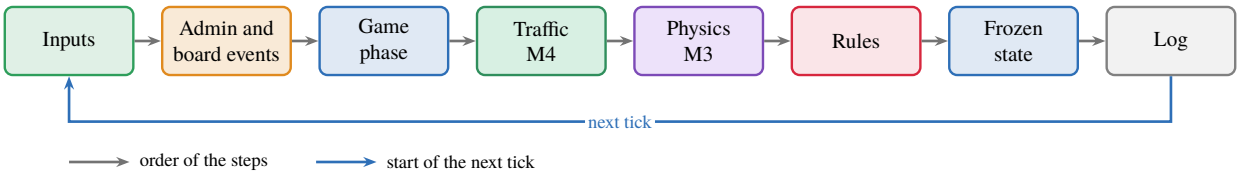
What a tick is:

The game world does not change all the time. It changes in small steps, like the pictures of a film. One step is called a tick, and the server makes 30 ticks in one second. Between two ticks, M2 only collects the messages from the phones, the admin and the board. In a tick, M1 uses all of them together, moves the world one step forward and makes a new state. Then M2 sends this state to everybody. So every player sees the same world after every tick.



The steps of one tick:

A tick always runs the same eight steps in the same order. First M1 takes the inputs that M2 has collected. Then it applies admin commands and board events, updates the game phase, asks M4 for traffic and bot controls, and asks M3 to move all motorcycles. After that the game rules run. At the end M1 builds a frozen copy of the state for M2 and writes one line to the game log. Because the order never changes and all random numbers come from one seed, the same inputs always give the same game.

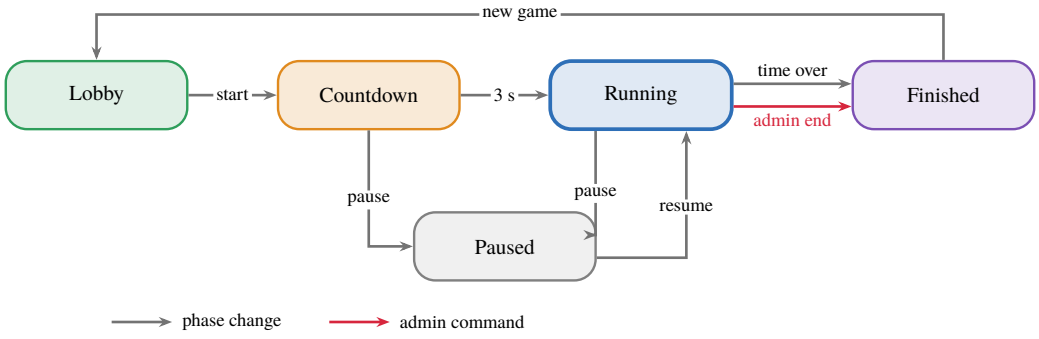


Controls inside a tick:

A phone sends its controls on every change and at least 10 times per second, but there are 30 ticks per second. So in some ticks a new control comes and in some ticks nothing comes. If nothing comes, M1 keeps the last control of that player. If more than one control comes, M1 uses the newest one for throttle, brake and steer. “Deliver” and “Find me” are different. They are events, not values. If any message in the tick has a “Deliver” press, the press counts, even when a newer message has no press. So a button press is never lost.

Game flow:

A game has five phases. In the lobby players join and choose a team. When the admin presses start, a 3 second countdown begins and nobody can move. Then the game runs for at most 5 minutes. The admin can end the game early. The admin can also pause and resume at any time. During a pause everything stops: the game time, the penalty times, the waiting time for lost players and the NPC cars. A pause during the countdown starts the countdown again after the resume. When the time ends, the team with the most points wins. If two teams have the same points, the game ends in a draw between them.



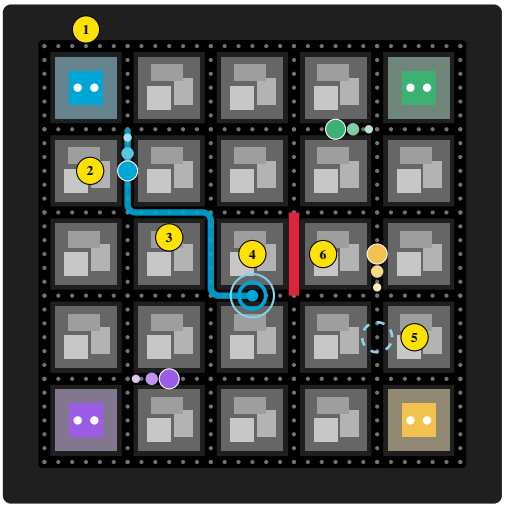
Players who leave and come back:

M2 sees when the link of a phone is lost and tells M1. M1 keeps the place of this player for 30 seconds. If the player does not come back in this time, a bot takes the motorcycle and plays on. If the player comes back later with the same player key, the bot gives the motorcycle back. The admin sets how many bots fill the empty places at the start.

Deliveries and points:

A player gets a package automatically when the motorcycle is inside the area of the own team's store. M1 then picks a delivery point for this player. M4 gives a table with the travel time from the store to every possible point. The times count the obstacles that are on the roads at that moment. M1 sorts this list and cuts it in the middle, so it gets a near half and a far half. The admin sets a "near chance" for each player, and M1 uses it to choose the half. Two players never get the same point at the same time. A far point gives more points. Every delivery gets a base value, and the longer the expected travel time is, the bigger the bonus on top of it. The admin sets both the base value and the bonus. The points are fixed when the point is chosen, so a new obstacle later does not change them.

To deliver, the player presses "Deliver". The button is always active. M1 accepts it only inside the delivery radius and below the speed limit. Both are admin parameters. If M1 says no, it sends the reason, for example "too fast" or "too far". A fall does not lose the package. The player only waits for the penalty time.



1 Team store in the team colour

2 Player with a package, a dot with a tail

3 Fastest route found by M4

4 Near delivery point, blinks in the player's colour

5 A far point, worth more points

6 Road work, so the route goes around

Earthquakes:

An earthquake can come from a hand hit on the board, from the vibration motors or from the admin button. All of them take the same path in M1. The board sends a strength from 0 to 10 and one of 9 zones. M1 uses the centre of that zone as the centre of the earthquake. A stronger hit makes a bigger earthquake, so the radius grows with the strength. The strongest hit reaches the largest radius, which the admin sets. Roads inside the radius get road work, and motorcycles inside the radius may fall. Events with a low confidence value are ignored. If the board is not connected, M1 makes the event itself when the admin presses the button, so the admin earthquake always works.

Obstacles and admin parameters:

M1 keeps one list of obstacles. Each obstacle has a type, a road segment, a lane and a position. The list has a version number, and the number goes up on every change. So M4 can see when it must compute routes again, and M5 and M8 can see when they must draw again. Every parameter has a name, a group, a minimum, a maximum and an owner module. When the admin changes a value, M1 checks the limits first. Then it saves the value, sends it to the owner (for example grip to M3) and sends "ok" back to the panel. A wrong value gets an error and the old value stays.

6. Testing

M1 is tested alone, without the network and without the real M2, M3 and M4. Small fake parts take their place inside the test and give fixed test data: scripted controls and joins, fixed poses and fall events, a fixed travel time table and recorded admin commands and board events. With this data we check every rule: the game phases with pause and end, the lobby and the bots, a lost player who comes back, the delivery cycle and the fair choice of points, two "Deliver" presses in one tick, every admin command with good and wrong values, and the earthquake radius. Because all random numbers come from one seed, the same test data must always give exactly the same game state.